CTF Write-Up - 1

January 30, 2026

CS 431

Computer and Network Security
Prof. Abhishek Bichhawat

Indian Institute of Technology Gandhinagar
Palaj, Gandhinagar - 382355

CTF 1 (UName: 22110089)

Submitted by

Guntas Singh Saran (22110089)
Computer Science and Engineering

Problem 1: Vigenère-101

Flag

`CNS431{v1g3n3r3_c1ph3r_1s_cl4ss1c}`

Understanding the Vulnerability

From the problem description and the server prompt, it was evident that the passcode was encrypted using the **Vigenère cipher**, and importantly, the encryption **key was already provided**.
The Vigenère cipher works by shifting each character of the plaintext using a repeating key. Each letter in the key corresponds to a Caesar shift value. For example, if the key letter is `V`, the plaintext letter is shifted by 21 positions. Decryption simply reverses this shift. The Vigenère cipher maps letters to numbers (A=0, B=1, ..., Z=25). For each character:

$$\mathrm{P}_i = (\mathrm{C}_i - \mathrm{K}_i) \bmod 26$$

where P_i is plaintext, C_i is ciphertext, and K_i is the repeated key character.

How I Exploited It

Since both the ciphertext and the key were given, the attack was straightforward decryption.

Steps Taken

1. Connecting to the challenge server using: `nc 10.0.118.104 5050`, the server displayed:
   - Key: `VICTORY`
   - Ciphertext: `Be3cmXfz!T0cEk4qb3bDb`
2. Having seen how the decryption works, I used an online implementation (**dcode.fr/vigenere-cipher** to apply the inverse shifts using the given key.
3. The decrypted plaintext was `Gw3atJob!Y0uCr4ck3dIt`.
4. Submitting this plaintext to the server returned the flag.

Problem 2: Columns-101

Flag

`THENUMBEROFRAILSFORTHERAILFENCECIPHERIS16`

Understanding the Vulnerability

This problem used a **Columnar Transposition cipher**. In this cipher, the plaintext is written row-wise into a grid and then read column-wise according to a secret column order. Importantly, the letters themselves are not changed - only the column positions are shuffled based on some order - usually the lexicographical order of the letters of the key.
The challenge stated that the **key length was 5**. This is a critical weakness because it limits the total number of possible column permutations to $5! = 120$, which is small enough to brute-force.

How I Exploited It

I tried all possible column orders of length 5 and checked which decrypted output looked like valid English - readable and meaningful

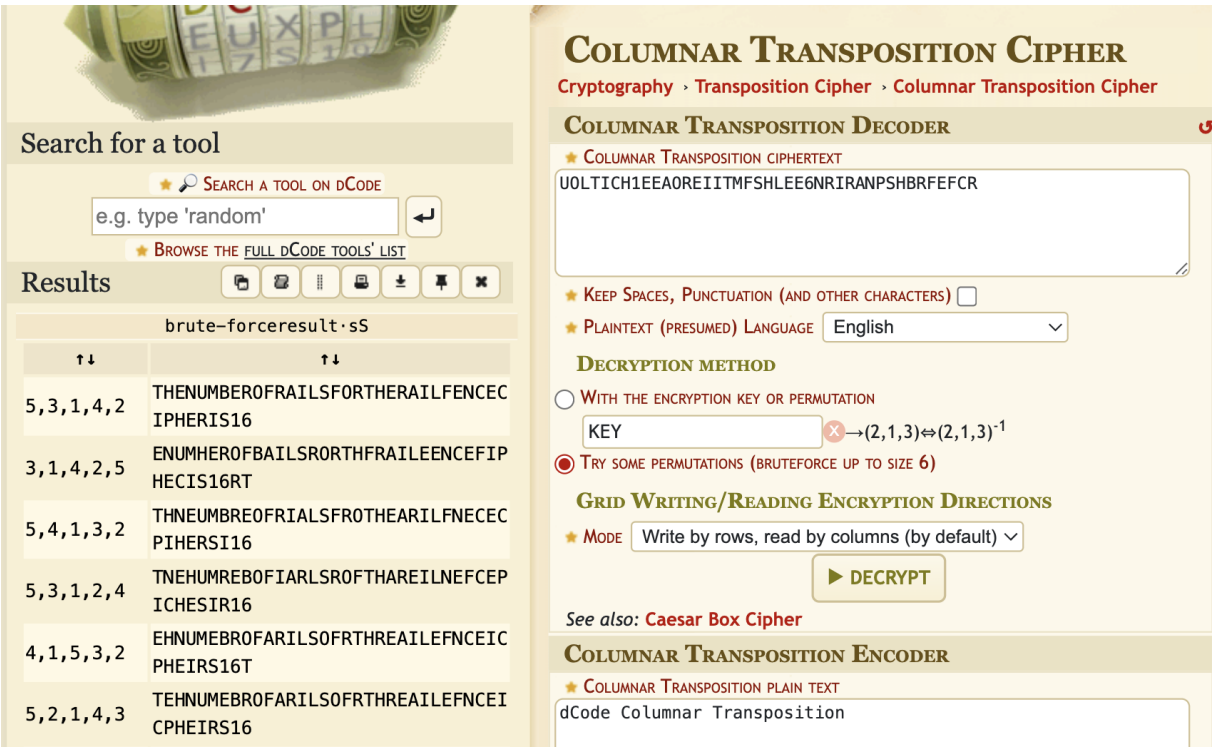

Figure 1: Various permutations of the columns with the first one making a meaningful statement.

Steps Taken

1. The cipher is identified as a columnar transposition based on the problem description.
2. Knowing the key length was 5, I used an online solver (dcode.fr/columnar-transposition-cipher) that enumerated all permutations of 5 columns.
3. The correct column order based on meaningful result was `[5,3,1,4,2]`, which produced: `THENUMBEROFRAILSFORTHERAILFENCECIPHERIS16`

Problem 3: Rail Force One

Flag

`MAYBETRYRAILS4AFTERTHISTCNMEASHIKWFUPVRLZGEUBOOJEOTEYOQRXDHD`

Understanding the Vulnerability

This ciphertext was encrypted using the **Rail Fence cipher** (as is clear from the title + the flag of previous statement), which writes the plaintext in a zig-zag pattern across a fixed number of rails and then reads it row by row.
The flag of problem 2 explicitly reveals that the number of rails used will be 16. Once the number of rails is known, the Rail Fence cipher can be deterministically decrypted.

For example, to decipher the message `DEZCDZGAOIG` and `N=3`, reconstruct the zigzag pattern with the expected number of levels.

```
X---X---X--
-X-X-X-X-X-
--X---X---X
```

Write the numbered letters in the pattern, line by line:

```
D---E---Z--
-C-D-Z-G-A-
--O---I---G
```

Read the letters along the zigzag path to reconstruct the original message `DCODEZIGZAG`.

How I Exploited It

Using the known number of rails, the zig-zag pattern can be reconstructed and the ciphertext mapped back onto it row by row.

Steps Taken

Having known the number of levels (16), it can easily constructed in the manner as shown above by hand, but I used an online implementation (dcode.fr/rail-fence-cipher) to do the same.

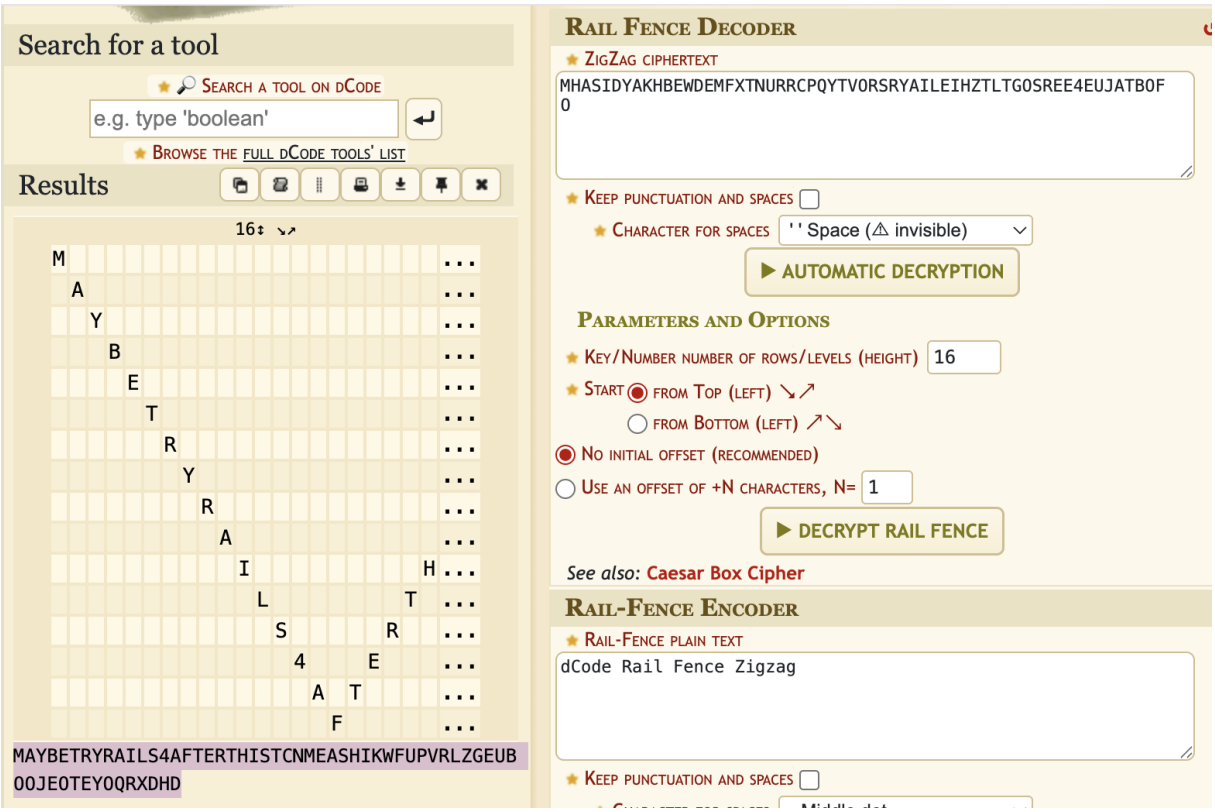

Figure 2: The simple decrypt of the rail fence cipher using the steps described above from the solver.

Problem 4: Rail Force Two

Flag

`THEQUICKBROWNFOXJUMPEDOVERTHELAZYDOGS`

Understanding the Vulnerability

This problem revealed that the previous Rail Fence encryption was **nested**. The problem explicitly instructed "*Remove the first 23 chars in the flag for Rail Force One and decrypt it again.*"

How I Exploited It

After removing the specified prefix, the remaining ciphertext was decrypted using Rail Fence with 4 rails - the previous flag again tells us to "*maybe try rails 4 after this*".

Steps Taken

Remove the first 23 characters from the previous plaintext to get ciphertext: `TCNMEASHIKWFUPVRLZGEUBOOJEOTEYOQRXDHD`. Using the same steps as problem 3 with 4 rails, we get `THEQUICKBROWNFOXJUMPEDOVERTHELAZYDOGS`.

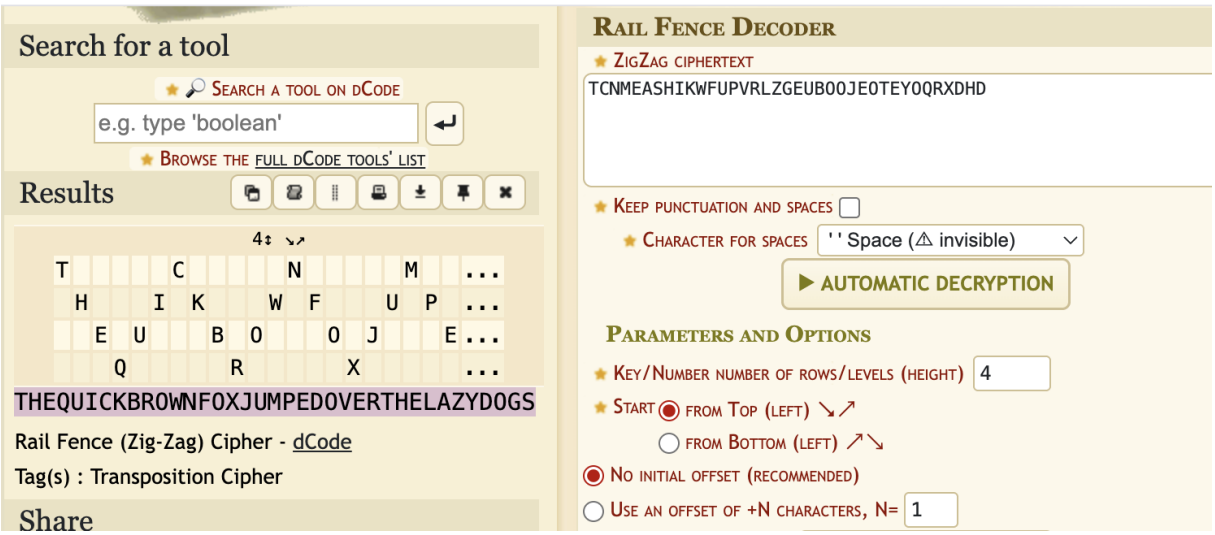

Figure 3: The simple decrypt of the rail fence cipher 2 using the steps described above from the solver.